UVM Verification of a

UART Core

Verification Plan

by

Hossam Shaher Anis

Cairo, 2026

Contents

Verification Plan 1
Contents 2
List of Tables 3
1 Introduction 4
2 Test Plan 5
3 Coverage Plan 7
4 Assertion Plan 8

List of Tables

Table 1 Test plan 5
Table 2 Coverage plan 7
Table 3 Assertion plan 8

1 Introduction

Serial communication uses a single data line (***serial line***) to exchange information between two systems.

The transmitting system converts the parallel data to a serial stream, and the receiving system reassembles the serial data back to its original parallel format.

A **UART (Universal Asynchronous Receiver and Transmitter) core** is the most commonly used scheme.

In this project, the UART core is verified using the SystemVerilog HDVL and the Universal Verification Methodology (UVM).

This document contains three plans:

1. Test plan.
2. Coverage plan.
3. Assertion (checking) plan.

2 Test Plan

Table 1 shows the test plan.

Table 1 Test plan

#[1]	Test Case Name[2]	Test Feature[3]	Sequence[4]
1	uart_test_random	Apply random stimuli to the baud rate generator, receiver (Rx), and transmitter (Tx). This test is completely random; no constraints are imposed.	Repeat the following sequence of actions `nu_repetitions` times: (1) Apply random stimuli to `reset` and `dvsr` (baud rate), then (2) Concurrently (using a `fork-join`), apply random stimuli to the receiver (Rx), and transmitter (Tx). Stimuli are completely random (not constrained).
2	uart_test_normal_operation	Same as uart_test_random *but* "normal operation" constraints are imposed.	Same sequence as in uart_test_random *but* "normal operation" constraints are imposed: (1) Baud rate doesn't change during simulation (always = 9600bps). (2) No reset occurs during transmitting/ receiving. (3) tx_start doesn't change during simulation (always = `1'b1`).
3	uart_test_dynamic_divisor	Same as uart_test_random *but* "dynamic divisor" constraints are imposed.	Same sequence as in uart_test_random *but* "dynamic divisor" constraints are imposed: (1) Baud rate changes during simulation (but not during transmitting/ receiving). Baud rates used in this test are 9600, 4800, and 2400 bps. (2) No reset occurs during transmitting/ receiving. (3) tx_start doesn't change during simulation (always = `1'b1`).
4	uart_test_directed_data	Same as uart_test_normal_operation *but* data to be transmitted/ received are directed using constraints.	Same sequence as in uart_test_normal_operation *but* data to be transmitted/ received are constrained to be `8'h00`, `8'hFF`, `8'h55`, or `8'hAA`. Each of these values is transmitted (by the Tx) and received (by the Rx) once.

[1] # = Serial number.
[2] Test case name = Test *scenario* name = UVM test name.
[3] Test feature = Objective of the scenario = Simple description of the scenario.
[4] Sequence = Detailed steps for stimulus generation for the scenario.

Table 1 Test plan (cont.)

#	Test Case Name	Test Feature	Sequence
5	uart_test_reset	Same as uart_test_random *but* "reset during normal operation" constraints are imposed.	Same sequence as in uart_test_random *but* "reset during normal operation" constraints are imposed: (1) Baud rate doesn't change during simulation (always = 9600bps). (2) A reset may occur during transmitting/ receiving. (3) tx_start doesn't change during simulation (always = `1'b1`). In uart_test_random two processes start in parallel to apply stimuli to TX and RX. In this test, a third process is added; it runs concurrently with those two to apply random stimuli to `reset` during transmitting/ receiving.

3 Coverage Plan

Table 2 shows the coverage plan.

Table 2 Coverage plan

#	Coverage Collector[5]	`covergroup/ cover property`	`coverpoint/ cross/ property`	Description
1	m_uart_coverage_collector_system	cover_group_system	dvsr	Value of `dvsr` is required to be 11, 23, and 47 (corresponding baud rates: 9600, 4800, and 2400 bps).
			reset	Value of `reset` is required to be 0 and 1.
2	m_uart_coverage_collector_rx	cover_group_rx	dout	Value of `dout_actual` is required to be `8'h00` (all zeros), `8'hFF` (all ones), and "`8'h55` or `8'hAA`" (alternating bits).
			dvsr	Same as `cover_group_system.dvsr`.
			reset	Same as `cover_group_system.reset`.
3	m_uart_coverage_collector_tx	cover_group_tx	tx	Value of `tx_actual` is required to be `8'h00` (all zeros), `8'hFF` (all ones), and "`8'h55` or `8'hAA`" (alternating bits) .
			dvsr	Same as `cover_group_system.dvsr`.
			reset	Same as `cover_group_system.reset`.
4	uart_rx_if_inst	UART_FRAME_RX_C	uart_frame_rx_p	The following behavior shall occur: At some clock tick, `rx_done_tick` shall be asserted (receiving process is completed), `rx` shall also be asserted (stop bit), and the past value of `rx` (before 9 baud periods) shall be de-asserted (start bit). I.e., a UART frame shall be received.
5	uart_tx_if_inst	UART_FRAME_TX_C	uart_frame_tx_p	The following behavior shall occur: At some clock tick, `tx_done_tick` shall be asserted (transmitting process is completed), `tx` shall also be asserted (stop bit), and the past value of `tx` (before 9 baud periods) shall be de-asserted (start bit). I.e., a UART frame shall be transmitted.

[5] Coverage collector = Class instance or interface instance

4 Assertion Plan

Table 3 shows the assertion plan.

Table 3 Assertion plan

#	Assertion ID	Description	File Name[6]
1	UART_FRAME_RX_A	At a certain clock tick, if `rx_done_tick` is asserted (receiving process is completed), `rx` shall also be asserted (stop bit), and the past value of `rx` (before 9 baud periods) shall be de-asserted (start bit).	uart_rx_if
2	UART_FRAME_TX_A	At a certain clock tick, if `tx_done_tick` is asserted (transmitting process is completed), `tx` shall also be asserted (stop bit), and the past value of `tx` (before 9 baud periods) shall be de-asserted (start bit).	uart_tx_if
3	STOP_BIT_RX_A	At the receiver (R_X), a start bit (`1'b0`) shall be followed by 8 data bits and a stop bit (`1'b1`) (iff no reset occurred during receiving).	uart_monitor_rx
4	DOUT_AT_RESET_A	At the receiver (R_X), when a reset occurs, `dout` shall be `'0`.	uart_monitor_rx
5	START_BIT_TX_A	At the transmitter (T_X), an assertion of `tx_start` shall be followed by a start bit (`1'b0`) (iff no reset occurred during transmitting).	uart_monitor_tx
6	STOP_BIT_TX_A	At the transmitter (T_X), a start bit (`1'b0`) shall be followed by 8 data bits and a stop bit (`1'b1`) (iff no reset occurred during transmitting).	uart_monitor_tx
7	TX_AT_RESET_A	At the transmitter (T_X), when a reset occurs, `tx` shall be `'0`.	uart_monitor_tx

[6] File Name = Interface/ monitor name